SUMMARY - BUILDING A BASIC RAG PIPELINE

By the end of this session, students will be able to:

1. **Differentiate** between core RAG variants (Memory, Branched, Adaptive, Corrective, and Self-RAG) and explain why each exists.
2. **Identify** which RAG strategy is most suitable for different real-world industry scenarios (e.g., personalization, comparisons, high-stakes compliance, cost optimization).
3. **Explain** how Memory RAG preserves user context across turns and why role-labeling of conversation history matters.
4. **Illustrate** how Branched RAG routes queries into different pipelines based on intent.
5. **Analyze** query difficulty signals and retrieval health metrics to justify Adaptive RAG design choices.
6. **Evaluate** draft answers for groundedness, coverage, and faithfulness, and describe how Corrective RAG repairs weak responses.
7. **Summarize** how Self-RAG introduces self-reflection, enabling the model to critique and improve its own drafts before responding.
8. **Compare and contrast** trade-offs among the variants in terms of latency, cost, accuracy, and reliability.
9. **Apply** the theory of RAG variants to design pipelines that are closer to industry-scale, production-ready systems.

Basic RAG Pipeline

In the previous session, we built embeddings for our documents (HTML headings, full text) and inserted them into Pinecone, with rich metadata (headings, keywords, timestamps). This gave us a **searchable vector index**.

But an index alone isn't useful unless we can **use it to answer questions**. This is where a **basic Retrieval-Augmented Generation (RAG) pipeline** comes in:

Pipeline Flow:

1. **User Query** → Encode into vector (same model).
2. **Retrieve** top-k results from Pinecone (with filters, e.g., by keyword metadata).
3. **Compose Prompt** → Insert retrieved context into a template.

4. **Call LLM (Gemini)** → Ask it to answer the user query **only from retrieved context**.

This transforms our static index into an **interactive knowledge assistant**.

1. From Index to Question Answering

Once documents are embedded and stored in Pinecone, the next step is to make the index useful. Retrieval by itself only returns fragments of documents. What we really want is a system that can take those fragments and **generate coherent answers**. A RAG pipeline does exactly this: it retrieves relevant pieces of text and passes them to an LLM, which then produces a grounded response.

2. Components of the Pipeline

2.1 Query Embedding and Retrieval

A user's query is encoded into a vector using the same embedding model as the documents. The vector database is then searched to find the most similar entries. These entries often carry metadata (headings, keywords, timestamps) which helps filter and refine results.

2.2 Prompt Construction

The retrieved context needs to be passed to the LLM in a clear, structured way. A simple prompt template asks the LLM to **answer strictly from the provided context** and admit when no answer is available. This ensures answers are reliable and not hallucinated.

2.3 LLM Generation

Finally, the LLM (Gemini in our case) receives the query and the retrieved context and produces a natural-language response. The generation step converts raw retrieval results into an answer that users can directly understand.

3. Implementation Walkthrough

3.1 Retrieval from Pinecone

```
query = "AI in simulations"
query_embedding = model.encode(query).tolist()

search_results = index.query(
    vector=query_embedding,
    top_k=5,
    include_values=False,
    include_metadata=True,
    filter={"heading_keywords": {"$in": ["ai"]}}
)

contexts = []
for match in search_results["matches"]:
    print(f"ID: {match['id']} | Score: {match['score']}")
    print(f"Metadata: {match['metadata']}")
    print("---")
    contexts.append(match['metadata'].get("headings", ""))
```

3.2 Prompt Template

```
def build_prompt(query, contexts):
    joined_context = "\n\n".join(contexts)
    return f"""
You are a helpful assistant. Use the following CONTEXT to answer the
QUESTION.
If the answer is not in the context, say "I don't know from the
provided documents."

QUESTION: {query}

CONTEXT:
{joined_context}
"""
```

3.3 Calling Gemini

```
import google.generativeai as genai

with open('gemini_api_key.txt', 'r') as f:
    gemini_key = f.read().strip()
genai.configure(api_key=gemini_key)

model_gemini = genai.GenerativeModel("gemini-1.5-flash")

prompt = build_prompt(query, contexts)
response = model_gemini.generate_content(prompt)

print("Answer:\n", response.text)
```

4. Key Takeaway

A basic RAG pipeline ties together **retrieval** and **generation**. The retriever ensures the LLM has access to relevant information, and the LLM transforms this raw context into coherent answers. This pipeline is the foundation upon which more advanced features like **memory**, **reranking**, and **adaptive retrieval** will be built.

Prompt Templating for RAG Pipelines

1. Introduction to Prompt Templates

1.1 What Prompt Templates Do

Prompt templates define a reusable structure for instructions given to the LLM. Instead of writing a new free-form prompt for every query, the template acts as a fixed frame with placeholders such as `{query}` and `{context}` that get filled dynamically at runtime.

1.2 Why They Are Important

Prompt templates bring consistency to outputs, reduce hallucinations, and make RAG systems easier to maintain. They allow engineers to manage prompts as a formal component of the pipeline, version them, and update them without changing the business logic.

2. Best Practices in Prompt Templating

2.1 Clear Structure

A good template includes role instruction, task instruction, the retrieved context, and output rules. It also defines how the LLM should behave when there is no relevant information in the context.

2.2 Storage and Management

Templates should not be hard-coded inside Python scripts. They are best stored in separate files in formats such as YAML, JSON, Markdown, or TXT. This separation makes them easier to review, update, and track in Git.

2.3 Industry Use

In production, templates are treated like source code. They are version-controlled, reviewed, and tested on evaluation datasets. Large organizations often maintain a prompt library or registry, tagging templates by use case, owner, and version.

3. Implementation Example

We now build a simple question answering system on top of our Pinecone index containing coffee-related webpages.

3.1 Retrieval from Pinecone

```
query = "What is Ashwagandha coffee?"
query_embedding = model.encode(query).tolist()

search_results = index.query(
    vector=query_embedding,
    top_k=5,
    include_values=False,
    include_metadata=True
)

contexts = []
for match in search_results["matches"]:
```

```
    contexts.append(match['metadata'].get("headings", ""))
```

3.2 Template File in YAML

```
name: coffee_rag_template
description: Template for answering coffee related questions
template: |
  You are a coffee expert assistant. Use the following CONTEXT to
answer the QUESTION.
  If the answer is not found, reply: "I don't know from the provided
documents."

  QUESTION: {query}

  CONTEXT:
  {context}
```

3.3 Python to Load and Use Template

```
import yaml, google.generativeai as genai

# Load YAML template
with open("prompts/coffee_rag_template.yaml", "r") as f:
    data = yaml.safe_load(f)
template = data["template"]

# Build prompt by filling placeholders
prompt = template.format(query=query, context="\n\n".join(contexts))

# Call Gemini
with open("gemini_api_key.txt", "r") as f:
    gemini_key = f.read().strip()
genai.configure(api_key=gemini_key)
model_gemini = genai.GenerativeModel("gemini-1.5-flash")
```

```
response = model_gemini.generate_content(prompt)
print("Answer:\n", response.text)
```

4. Key Takeaway

Prompt templates transform retrieval results into structured inputs for the LLM. They enforce consistency, allow maintainability through external storage and version control, and make RAG pipelines production-ready. This sets the foundation for more advanced strategies like memory-based RAG and corrective RAG.

LLM Calls and Memory in RAG Pipelines

1. Moving from Retrieval to Generation

Once relevant passages have been retrieved from the vector database, the system must pass them to the language model. The raw retrieved text alone is not enough, because the model needs structured instructions about how to use it. This is where prompt templates play a critical role. A prompt template acts as a scaffold into which both the current user query and the retrieved context are placed. The model then produces a grounded answer based on this combined input.

The act of calling the LLM with retrieved context is therefore the core of the basic RAG pipeline: a user query is embedded, the vector database returns matching passages, these passages are inserted into a template, and the template is then given to the model.

2. Prompt Templates as the Bridge

Prompt templates ensure that the retrieved content is interpreted consistently. They also provide rules on how the model should answer, for example instructing it to only use retrieved evidence, to keep answers concise, or to admit when the information is missing. Without such a template, the model may blend retrieved content with its own prior knowledge, which often results in hallucinations.

A simple template might read:

```
You are a knowledgeable assistant. Use the CONTEXT provided to answer
the QUESTION.
If the answer is not present, reply: "I don't know from the provided
documents."

Conversation so far:
{history}

QUESTION: {query}

CONTEXT:
{context}
```

Here the `{context}` placeholder is filled with the retrieved passages, `{query}` is the latest user question, and `{history}` is the accumulated conversation. This shows how the same template can unify retrieval, query, and memory in one structure.

3. Incorporating Memory in RAG

A single-turn RAG system can answer questions accurately, but conversations require memory. When the user asks a follow-up such as “Compare it with turmeric latte,” the model must remember that “it” refers to Ashwagandha coffee. To achieve this, the conversation history is maintained as text and is added to the template at every turn.

The memory can be understood in two parts. Short-term memory is the ongoing conversation itself, which is re-sent in every prompt under the label “Conversation so far.” Long-term memory can be implemented by storing facts or past user preferences in a vector database, so that they can be retrieved just like documents.

4. Differentiating User and Assistant Text

When sending conversation history back to the LLM, it is important to mark which lines came from the user and which came from the assistant. This prevents the model from confusing its own earlier outputs with new instructions. A simple format is:

```
User: Tell me about Ashwagandha coffee.
```

```
Assistant: Ashwagandha coffee is a modern adaptation...
User: Compare it with turmeric latte.
```

By clearly separating roles, the LLM can treat previous answers as part of the conversation rather than new commands. This explicit labeling is the foundation of dialogue memory.

5. Re-querying the Vector Database at Each Step

Even though the conversation history is passed along, the vector database is always queried again with the latest user input. This ensures that the model grounds each answer in relevant external knowledge, not only in the dialogue. The pipeline at every turn therefore follows this cycle:

1. Encode the latest user query into an embedding.
2. Retrieve top-k passages from the vector database.
3. Combine retrieved passages with conversation history inside the prompt template.
4. Call the LLM with the full prompt.
5. Output the answer and append it to the conversation history.

This cycle repeats for every question, which means memory (conversation history) and retrieval (fresh documents) are always used together.

6. Example Implementation

```
# Example conversation history
history = """
User: Tell me about Ashwagandha coffee.
Assistant: Ashwagandha coffee is a modern adaptation that blends...
"""

# New query
query = "Compare it with turmeric latte"
query_embedding = model.encode(query).tolist()
```

```
# Retrieve relevant docs
search_results = index.query(
    vector=query_embedding,
    top_k=5,
    include_values=False,
    include_metadata=True
)
contexts = [m['metadata'].get("headings", "") for m in
search_results["matches"]]

# Build prompt
def build_prompt(query, contexts, history):
    return f"""
You are a coffee expert assistant. Use the CONTEXT to answer the
QUESTION.
If the answer is not in the context, say "I don't know from the
provided documents."

Conversation so far:
{history}

QUESTION: {query}

CONTEXT:
{"\n\n".join(contexts)}
"""

prompt = build_prompt(query, contexts, history)

# Call Gemini
response = model_gemini.generate_content(prompt)
print("Answer:\n", response.text)
```

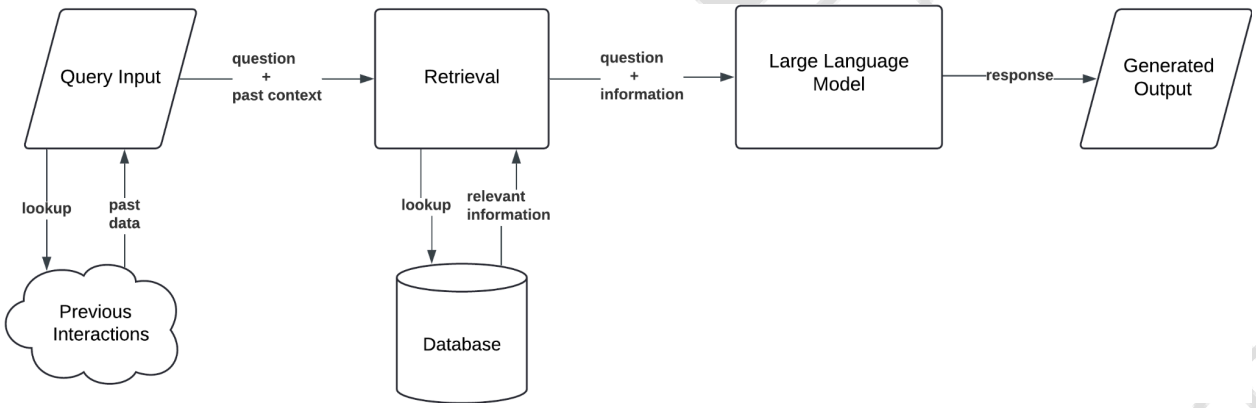
7. Key Understanding

Calling the LLM with retrieved context is the heart of a RAG system. Adding memory makes the pipeline conversational and context-aware. Prompt templates serve as the bridge, ensuring that user queries, assistant answers, retrieved documents, and conversation history are clearly separated and consistently interpreted. At every turn, the system relies both on the vector database for grounding and on structured memory for continuity.

Memory-based RAGs

Memory-based RAG combines two kinds of context on every turn: retrieved knowledge from a vector database and conversational memory from prior turns. Short-term memory is the rolling dialogue history; long-term memory is a store of distilled user facts and preferences (e.g., “prefers low-caffeine drinks”), typically kept as embeddings in a separate namespace or index. Each user message triggers a fresh retrieval against documents and, optionally, a retrieval against long-term memory. The prompt template keeps these sources separated and labeled so the model can reason correctly.

The cycle at every turn is consistent. Encode the latest user query. Retrieve top-k from the documents namespace. Retrieve top-m from the long-term memory namespace. Construct a prompt that includes Conversation so far, Relevant long-term memory, and Retrieved context. Call the LLM to generate an answer grounded in context while respecting memory. Append the new turn to short-term memory and optionally extract and store new long-term facts.



1. Goal

1.1 We will build a conversational, memory-based RAG that:

- retrieves from your **coffee pages** index on every turn,
- maintains **short-term memory** (conversation history),
- stores **long-term memory** (user facts/preferences) in a separate Pinecone namespace,
- uses a **prompt template** loaded from a YAML file (kept out of code),
- runs an **interactive `input()` loop** so students can keep asking questions.

2. Project layout

2.1 Create a simple structure so prompts live outside code.

```
project/
  prompts/
    coffee_memory_rag.yaml
  api_key.txt
  gemini_api_key.txt
  notebook.ipynb  ← put the cells below here
  coffee_pages/   ← your HTML files (or point to your Windows path)
```

3. Prompt template (kept outside code)

3.1 Create `prompts/coffee_memory_rag.yaml` with a clear, reusable template.

We keep role, task, refusal, and where memory and evidence go. The LLM will **only** use CONTEXT and may personalize using RELEVANT_MEMORY.

```
name: coffee_memory_rag
description: Prompt for a memory-based RAG over coffee documents
template: |
  You are a helpful coffee expert. Use CONTEXT and, when useful,
RELEVANT_MEMORY to answer the QUESTION.
  If the answer is not in CONTEXT, reply exactly: "I don't know from
the provided documents."
  Do not invent sources. Cite sources by their [DocID] when you use
them.
```

```
  Conversation so far:
  {history}

  Relevant memory (facts about this user):
  {memory_block}

  QUESTION:
  {query}

  CONTEXT (retrieved docs):
  {context_block}
```

4. Setup and keys

We import required libraries, then read Pinecone and Gemini keys from files so we never hardcode secrets into the notebook.

```
# !pip install sentence-transformers pinecone-client
google-generativeai beautifulsoup4

from pathlib import Path
import time, os, re, json
from typing import List, Dict, Tuple
from bs4 import BeautifulSoup

from sentence_transformers import SentenceTransformer
from pinecone import Pinecone, ServerlessSpec
import google.generativeai as genai
```

```
# Read Pinecone and Gemini API keys from local files.
# This keeps secrets out of source control and lets you rotate them
easily.
with open('api_key.txt', 'r') as f:
    PINECONE_API_KEY = f.read().strip()
```

```
with open('gemini_api_key.txt','r') as f:
    GEMINI_API_KEY = f.read().strip()
```

5. Initialize services and models

5.1 We connect to Pinecone with your API key.

5.2 We create or open the index named `coffeeindex` with cosine similarity and dimension `384` (the size for `paraphrase-MiniLM-L6-v2`).

5.3 We create two namespaces: `docs` for your coffee articles, and `mem_{user_id}` for per-user memory.

5.4 We load the embedding model and Gemini LLM.

```
# Initialize Pinecone client
pc = Pinecone(api_key=PINECONE_API_KEY)

INDEX_NAME = "coffeeindex"
REGION = "us-east-1"
DIM = 384

# Create an index if it doesn't exist (safe to call, we first list
indexes)
existing = [d["name"] for d in pc.list_indexes()]
if INDEX_NAME not in existing:
    pc.create_index(
        name=INDEX_NAME,
        dimension=DIM,
        metric='cosine',
        spec=ServerlessSpec(cloud='aws', region=REGION)
    )

# Get a handle to the index
index = pc.Index(INDEX_NAME)

# Namespaces: docs for content; mem_user for long-term memory
DOCS_NS = "docs"
```

```
USER_ID = "user_001"
MEM_NS  = f"mem_{USER_ID}"

# Embedding model (384-dim)
EMBED_MODEL = "paraphrase-MiniLM-L6-v2"
embedder = SentenceTransformer(EMBED_MODEL)

# Gemini LLM
genai.configure(api_key=GEMINI_API_KEY)
llm = genai.GenerativeModel("gemini-1.5-flash")
```

6. Load the prompt template from YAML

6.1 We read the YAML file at runtime so content designers can tweak language without touching code.
6.2 We'll later do `.format()` to fill `{history}`, `{memory_block}`, `{query}`, `{context}`.

```
import yaml

def load_prompt_template(path: str) -> str:
    with open(path, "r", encoding="utf-8") as f:
        data = yaml.safe_load(f)
    return data["template"]

PROMPT_TEMPLATE =
load_prompt_template("prompts/coffee_memory_rag.yaml")
```

7. Ingest HTML pages into the docs namespace

7.1 We point to your folder of HTML coffee pages. On Windows you might use your original path; here we keep it variable so this notebook runs anywhere.
7.2 For each HTML file:

- parse and strip text with BeautifulSoup,
- generate an embedding once for the **whole page** (simple for class; in production you should chunk),
- attach helpful metadata: file_name, title (first heading), concatenated headings, extracted heading keywords, timestamp,
- upsert `(id, vector, metadata)` to Pinecone under `docs`.

```
# Adjust this path to where your HTML files are.
# Example 1 (Windows): Path(r"D:\Sameer\descipr\webinars\vector database\coffee_pages")
# Example 2 (colab/local): Path("coffee_pages")
html_dir = Path("coffee_pages")

def extract_heading_keywords(headings: List[str]) -> List[str]:
    # basic tokenization for demo; in prod, normalize, drop stopwords, etc.
    toks = []
    for h in headings:
        toks += re.findall(r"[A-Za-z][A-Za-z\-]+", h)
    return sorted({t.lower() for t in toks})

def upsert_docs_from_html(directory: Path) -> int:
    pairs = []
    for fp in directory.glob("*.html"):
        raw = fp.read_text(encoding="utf-8")
        soup = BeautifulSoup(raw, "html.parser")

        # 1) Extract readable text
        page_text = soup.get_text(" ", strip=True)

        # 2) Build an embedding vector
        vec = embedder.encode([page_text], normalize_embeddings=True)[0].tolist()

        # 3) Derive metadata for later filtering and citations
        headings = [h.get_text(" ", strip=True) for h in
```

```
soup.find_all(['h1','h2','h3','h4','h5','h6'])]
        meta = {
            "file_name": fp.name,
            "title": headings[0] if headings else fp.stem,
            "headings": " | ".join(headings[:12]) if headings else
"",
            "heading_keywords": extract_heading_keywords(headings),
            "ts": int(time.time())
        }

        # 4) Prepare Pinecone vector tuple (id, vector, metadata)
        doc_id = fp.stem
        pairs.append((doc_id, vec, meta))

    if pairs:
        index.upsert(vectors=pairs, namespace=DOCS_NS)
    return len(pairs)

indexed = upsert_docs_from_html(html_dir)
print(f"Indexed {indexed} HTML pages into namespace '{DOCS_NS}'.")
```

8. Memory extraction and storage

8.1 We implement a **very simple** extractor that captures lines like "I prefer…", "I avoid…", "My…".

8.2 Each extracted fact becomes a vector with metadata `{fact, user_id, ts}` and is stored in the memory namespace.

8.3 We also provide a memory retriever that queries the memory namespace.

```
def extract_user_facts(user_text: str) -> List[str]:
    """
    Demo-only extractor. In production:
      - use an LLM-based information extractor,
      - define a strict JSON schema for {preference, value, evidence,
ts}.
```

```
    """
    pats = [
        r"\bI (?:like|love|prefer)\b[^.]+",
        r"\bI (?:avoid|usually|often|am)\b[^.]+",
        r"\bMy [A-Za-z ]+\b[^.]+"
    ]
    findings = []
    for pat in pats:
        findings += [m.group(0).strip() for m in re.finditer(pat,
user_text, flags=re.I)]
    return sorted(set(findings))

def add_memory_facts(facts: List[str]) -> None:
    if not facts:
        return
    vecs = embedder.encode(facts, normalize_embeddings=True).tolist()
    now = int(time.time())
    payload = []
    for i, fact in enumerate(facts):
        vid = f"{USER_ID}:{now}:{i}"
        meta = {"fact": fact, "user_id": USER_ID, "ts": now}
        payload.append((vid, vecs[i], meta))
    index.upsert(vectors=payload, namespace=MEM_NS)

def retrieve_memory(query_text: str, k: int = 3) -> List[Dict]:
    qv = embedder.encode([query_text],
normalize_embeddings=True)[0].tolist()
    res = index.query(
        vector=qv, top_k=k, include_values=False,
include_metadata=True, namespace=MEM_NS
    )
    return res.get("matches", [])
```

9. Retrieval from docs

9.1 We embed the latest user turn, then query the docs namespace.
9.2 Optional: pass a list of keyword filters to restrict by headings metadata (simple demo of metadata filtering).

```
def retrieve_docs(query_text: str, k: int = 5, keywords: List[str] =
None) -> List[Dict]:
    qv = embedder.encode([query_text],
normalize_embeddings=True)[0].tolist()
    filt = {"heading_keywords": {"$in": [kw.lower() for kw in
keywords]}} if keywords else None
    res = index.query(
        vector=qv, top_k=k, include_values=False,
include_metadata=True,
        namespace=DOCS_NS, filter=filt
    )
    return res.get("matches", [])
```

10. Formatting helpers and prompt assembly

10.1 The LLM needs to clearly see who said what, what memory applies, and what documents are being cited.
10.2 We format conversation history with explicit roles “User:” and “Assistant:”.
10.3 We compress doc hits to a neat list like
`[03_turmeric_coffee_haldi_cappuccino] Turmeric Coffee – …`
10.4 We fill the YAML template placeholders and return the final prompt string.

```
def format_history(history: List[Dict]) -> str:
    if not history: return "None"
    return "\n".join(
        f"{'User' if t['role']=='user' else 'Assistant'}:
{t['content']}"
        for t in history
    )

def format_memory(mem_hits: List[Dict]) -> str:
    if not mem_hits: return "None"
    return "\n".join(f"- {h['metadata'].get('fact','')}" for h in
```

```
mem_hits)

def format_context(doc_hits: List[Dict]) -> str:
    if not doc_hits: return "None"
    lines = []
    for h in doc_hits:
        doc_id = h.get("id", "")
        meta = h.get("metadata", {})
        title = meta.get("title") or meta.get("file_name") or
"Untitled"
        lines.append(f"[{doc_id}] {title}")
    return "\n".join(lines)

def build_prompt(query: str, history: List[Dict], doc_hits:
List[Dict], mem_hits: List[Dict]) -> str:
    return PROMPT_TEMPLATE.format(
        history=format_history(history),
        memory_block=format_memory(mem_hits),
        query=query,
        context_block=format_context(doc_hits)
    )
```

11. One conversational turn (RAG + memory)

On each user message we:

- extract and store new memory facts from the user's text,
- retrieve from docs and memory namespaces,
- build a prompt that includes history, memory, and doc citations,
- call the LLM,
- append both user and assistant turns to history,
- print a compact trace so students can see what was retrieved and used.

```
history: List[Dict] = []
```

```
def rag_turn(user_text: str, k_docs: int = 5, k_mem: int = 3,
keywords: List[str] = None) -> str:
    # 1) long-term memory update from user message
    new_facts = extract_user_facts(user_text)
    add_memory_facts(new_facts)

    # 2) retrieve current docs + memory
    doc_hits = retrieve_docs(user_text, k=k_docs, keywords=keywords)
    mem_hits = retrieve_memory(user_text, k=k_mem)

    # 3) assemble the prompt from template
    prompt = build_prompt(user_text, history, doc_hits, mem_hits)

    # 4) call the LLM
    resp = llm.generate_content(prompt)
    answer = getattr(resp, "text", "").strip()

    # 5) update short-term memory (conversation history)
    history.append({"role": "user", "content": user_text})
    history.append({"role": "assistant", "content": answer})

    # 6) tracing for teaching
    print("---- DOCS ----")
    for h in doc_hits:
        print(h["id"], round(h["score"],3), "--",
h["metadata"].get("title") or h["metadata"].get("file_name"))
    print("---- MEMORY ----")
    for h in mem_hits:
        print("-", h["metadata"]["fact"])
    print("---- ANSWER ----")
    print(answer, "\n")

    return answer
```

12. Interactive loop with `input()`

12.1 Students can chat live with the assistant.
12.2 Type `exit` to stop.
12.3 Try a sequence that demonstrates memory:

- "Tell me about ashwagandha coffee in India."
- "I prefer low-caffeine drinks and usually avoid very bitter flavors."
- "Suggest a coffee for me accordingly and compare it with turmeric latte."

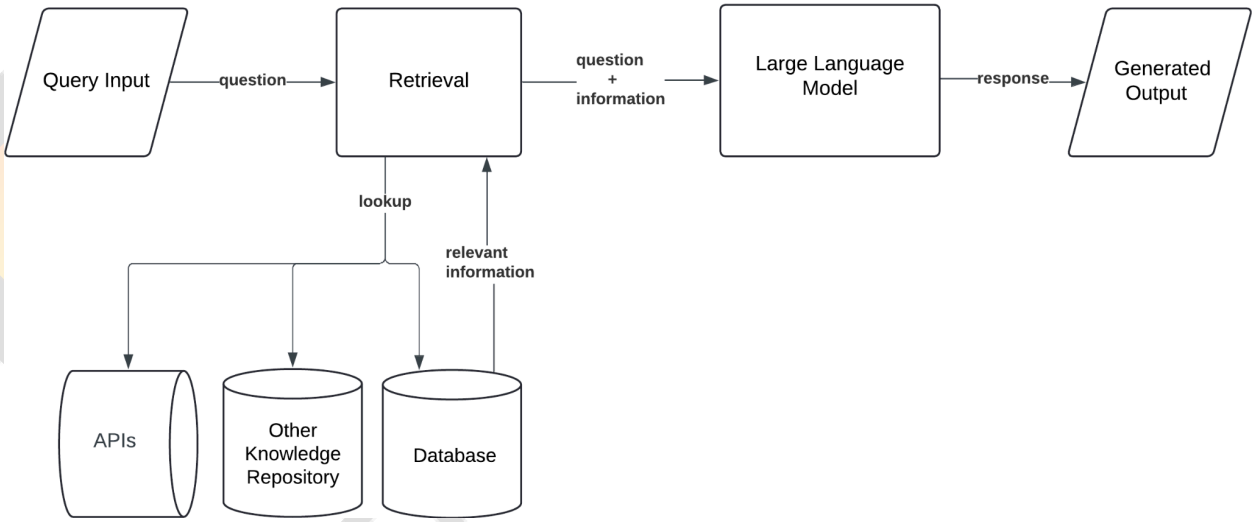
```
print("Coffee Memory-RAG. Type 'exit' to quit.\n")

while True:
    q = input("You: ").strip()
    if q.lower() in {"exit","quit"}:
        print("Goodbye!")
        break
    _ = rag_turn(q)
```

13. Extensions for homework

14.1 **Chunking and snippets**: split pages into smaller chunks and store short text snippets in metadata; show the exact snippet beneath each citation.
14.2 **LLM-based memory extraction**: replace the regex extractor with a small Gemini prompt that writes `{preference, value, evidence, ts}` JSON.
14.3 **History summarization**: after N turns, roll the history into a compact summary to stay within context limits.
14.4 **Caching**: memoize frequent queries and their retrieval results to reduce latency and cost.

Branched RAG



1. Concept

1.1 Why branch

One retrieval recipe rarely fits every question. Queries that ask for short definitions, personal recommendations, or A–vs–B comparisons benefit from **different retrieval settings and prompts**. Branching routes each query to a specialized mini-pipeline so answers are more precise, shorter, and cheaper.

1.2 Branches and signals

We'll use lightweight, explainable signals (no extra model needed) to pick a branch.

- **Explain**: "what is…", "tell me about…", "benefits of…"
- **Compare**: "compare X with Y", "X vs Y"
- **Recommend (uses memory)**: "suggest… for me", "what should I try…", "I prefer…"
- **Lookup (keyword/exact)**: numbers, ratios, brew times; heavy use of metadata keywords

1.3 Turn flow (for every user message)

- Detect branch from the query text.
- Retrieve documents with branch-specific parameters (k, filters).
- For "recommend", also retrieve long-term **memory** and include it.
- Build a **branch-specific prompt** (templates live in separate YAML files).
- Call the LLM, return answer, update conversation history (and memory if new facts appear).

- If retrieval is weak (few/low-score hits), **fallback** to a safer branch (e.g., Explain → Lookup).

2. Code (Router and Retrieval)

Assumptions: your Pinecone `index`, `embedder`, `llm`, `DOCS_NS`, `MEM_NS`, and `history` already exist from earlier notebooks. Prompts live in `prompts/` as separate YAML files (shown below). Memory helpers (`extract_user_facts`, `add_memory_facts`, `retrieve_memory`) can be reused from your Memory-RAG notebook.

2.1 Router (simple, explainable rules)

Before the code: we implement a tiny function that inspects the query and returns one of `{"explain","compare","recommend","lookup"}`. This keeps routing transparent for students.

```
import re

def route_query(q: str) -> str:
    ql = q.lower()

    # 1) compare signals
    if re.search(r"\bvs\b|compare|difference between", ql):
        return "compare"

    # 2) recommend/personal signals
    if re.search(r"\bsuggest|recommend|for me|i prefer|i like|what
should i try", ql):
        return "recommend"

    # 3) lookup-ish: numbers/ratios/times/temperatures/percentages
    if re.search(r"\b\d+
?(mg|ml|g|%|minutes|min|ratio|:|°c|celsius)\b", ql):
        return "lookup"

    # 4) default explain
    if re.search(r"\bwhat is|tell me about|benefits of|overview
of|explain\b", ql):
```

```
        return "explain"

    # safe default
    return "explain"
```

2.2 Retrieval helpers per branch

Before the code: each branch tweaks `k` and optional metadata filters. We keep it simple and dense-only (Pinecone) to avoid new infra; the **Lookup** branch also tries a small keyword filter using the `heading_keywords` metadata we stored at ingest.

```
def embed(text: str):
    return embedder.encode([text],
normalize_embeddings=True)[0].tolist()

def retrieve_docs(query_text: str, k: int = 5, keywords=None):
    vec = embed(query_text)
    filt = {"heading_keywords": {"$in": [kw.lower() for kw in
keywords]}} if keywords else None
    res = index.query(
        vector=vec, top_k=k, include_values=False,
include_metadata=True,
        namespace=DOCS_NS, filter=filt
    )
    return res.get("matches", [])

def retrieve_for_explain(q: str):
    # concise, narrative passages
    return retrieve_docs(q, k=5)

def retrieve_for_compare(q: str):
    # broader context to cover both items
    return retrieve_docs(q, k=10)

def retrieve_for_recommend(q: str):
    # keep doc k modest; personalization will come from memory
    return retrieve_docs(q, k=6)
```

```
def retrieve_for_lookup(q: str):
    # try to catch exact tokens that were saved from headings
    # extract simple alpha tokens as "keywords"
    kws = re.findall(r"[A-Za-z][A-Za-z\-]+", q)
    hits = retrieve_docs(q, k=8, keywords=kws[:6])  # cap keyword
list
    if not hits:
        # fallback to plain dense
        hits = retrieve_docs(q, k=8)
    return hits
```

3. Code (Prompts, Orchestrator, Demo)

3.1 Load branch-specific templates (from YAML files)

Before the code: we keep prompts out of code for maintainability. Create these files:

prompts/branch_explain.yaml

```
name: branch_explain
template: |
  You are a coffee expert. Answer the QUESTION using only CONTEXT. If
missing, say:
  "I don't know from the provided documents." Cite sources as
[DocID].

  Conversation so far:
  {history}

  QUESTION:
  {query}

  CONTEXT:
  {context_block}
```

prompts/branch_compare.yaml

```
name: branch_compare
template: |
  You are a coffee expert. Compare the two items in the QUESTION
using only CONTEXT.
  Structure as short bullets: similarities, differences, and a brief
verdict.
  If info is missing, say so. Cite sources as [DocID].

  Conversation so far:
  {history}

  QUESTION:
  {query}

  CONTEXT:
  {context}
```

prompts/branch_recommend.yaml

```
name: branch_recommend
template: |
  You are a coffee expert. Make a personalized recommendation based
on RELEVANT_MEMORY,
  supported by CONTEXT. If insufficient, ask a brief clarifying
question.
  Cite sources as [DocID].

  Conversation so far:
  {history}

  Relevant memory (facts about this user):
  {memory_block}

  QUESTION:
  {query}
```

```
  CONTEXT:
  {context}
```

prompts/branch_lookup.yaml

```
name: branch_lookup
template: |
  You are a precise assistant. Extract the specific fact(s) asked in
the QUESTION
  using only CONTEXT. If not present, reply exactly:
  "I don't know from the provided documents." Cite sources as
[DocID].
  Keep it under 80 words.

  Conversation so far:
  {history}

  QUESTION:
  {query}

  CONTEXT:
  {context}
```

Now load them:

```
import yaml, os

TEMPLATE_PATHS = {
    "explain":   "prompts/branch_explain.yaml",
    "compare":   "prompts/branch_compare.yaml",
    "recommend": "prompts/branch_recommend.yaml",
    "lookup":    "prompts/branch_lookup.yaml",
}

def load_template(branch: str) -> str:
    with open(TEMPLATE_PATHS[branch], "r", encoding="utf-8") as f:
        return yaml.safe_load(f)["template"]
```

3.2 Formatting helpers for history, memory, and context

Before the code: we re-use your role-labeled history format; context lines show `[DocID] Title`. Recommendation branch can include memory; other branches pass `"None"` there.

```
def format_history(history):
    if not history: return "None"
    return "\n".join(f"{'User' if h['role']=='user' else
'Assistant'}: {h['content']}" for h in history)

def format_context(hits):
    if not hits: return "None"
    lines = []
    for h in hits:
        doc_id = h.get("id","")
        meta = h.get("metadata",{})
        title = meta.get("title") or meta.get("file_name") or
"Untitled"
        lines.append(f"[{doc_id}] {title}")
    return "\n".join(lines)

def format_memory(mem_hits):
    if not mem_hits: return "None"
    return "\n".join(f"- {h['metadata'].get('fact','')}" for h in
mem_hits)
```

3.3 Branch-specific prompt builders

```
def build_prompt_explain(query, history, doc_hits):
    tpl = load_template("explain")
    return tpl.format(
        history=format_history(history),
        query=query,
        context_block=format_context(doc_hits)
    )

def build_prompt_compare(query, history, doc_hits):
    tpl = load_template("compare")
```

```
    return tpl.format(
        history=format_history(history),
        query=query,
        context_block=format_context(doc_hits)
    )

def build_prompt_recommend(query, history, doc_hits, mem_hits):
    tpl = load_template("recommend")
    return tpl.format(
        history=format_history(history),
        query=query,
        context_block=format_context(doc_hits),
        memory_block=format_memory(mem_hits)
    )

def build_prompt_lookup(query, history, doc_hits):
    tpl = load_template("lookup")
    return tpl.format(
        history=format_history(history),
        query=query,
        context_block=format_context(doc_hits)
    )
```

3.4 Orchestrator (one function to rule the branches)

Before the code: the orchestrator picks a branch, runs the matching retrieval, assembles the prompt, calls the LLM, updates history, and (optionally) updates memory if the user states new preferences.

```
# Reuse these from your Memory-RAG notebook if available
def extract_user_facts(txt: str):
    pats = [r"\bI (?:like|love|prefer)\b[^.]+", r"\bI
(?:avoid|usually|often|am)\b[^.]+", r"\bMy [A-Za-z ]+\b[^.]+"]
    out = []
    for pat in pats:
        out += [m.group(0).strip() for m in re.finditer(pat, txt,
```

```
flags=re.I)]
    return sorted(set(out))

def add_memory_facts(facts: list):
    if not facts: return
    vecs = embedder.encode(facts, normalize_embeddings=True).tolist()
    now = int(time.time())
    payload = []
    for i, fact in enumerate(facts):
        vid = f"{'user_001'}:{now}:{i}"
        payload.append((vid, vecs[i], {"fact": fact,
"user_id":"user_001", "ts": now}))
    index.upsert(vectors=payload, namespace=MEM_NS)

def retrieve_memory(q: str, k: int = 3):
    vec = embed(q)
    res = index.query(vector=vec, top_k=k, include_values=False,
include_metadata=True, namespace=MEM_NS)
    return res.get("matches", [])

# Global history
history = []

def branched_rag_turn(user_text: str) -> str:
    # 1) branch selection
    branch = route_query(user_text)

    # 2) store new user memory (only from user text)
    add_memory_facts(extract_user_facts(user_text))

    # 3) branch-specific retrieval
    if branch == "explain":
        doc_hits = retrieve_for_explain(user_text)
        prompt = build_prompt_explain(user_text, history, doc_hits)

    elif branch == "compare":
        doc_hits = retrieve_for_compare(user_text)
        prompt = build_prompt_compare(user_text, history, doc_hits)
```

```
    elif branch == "recommend":
        doc_hits = retrieve_for_recommend(user_text)
        mem_hits = retrieve_memory(user_text, k=3)
        prompt = build_prompt_recommend(user_text, history, doc_hits,
mem_hits)

    elif branch == "lookup":
        doc_hits = retrieve_for_lookup(user_text)
        prompt = build_prompt_lookup(user_text, history, doc_hits)

    else:
        # safe fallback
        doc_hits = retrieve_for_explain(user_text)
        prompt = build_prompt_explain(user_text, history, doc_hits)

    # 4) LLM call
    resp = llm.generate_content(prompt)
    answer = getattr(resp, "text", "").strip()

    # 5) update rolling history
    history.append({"role":"user", "content": user_text})
    history.append({"role":"assistant", "content": answer})

    # 6) small trace
    print(f"[Branch: {branch}]  Top docs: {[h['id'] for h in
doc_hits[:3]]}")
    print(answer, "\n")
    return answer
```

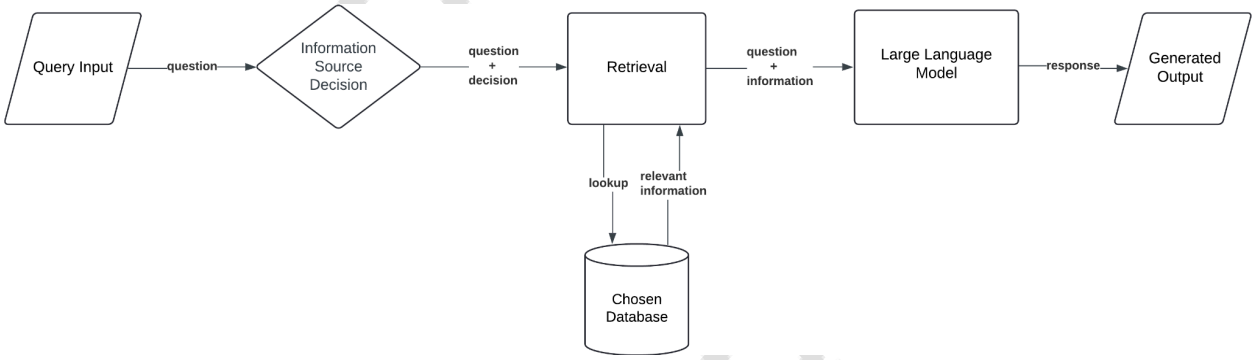
3.5 Quick demo loop

```
print("Branched RAG -- try queries like:\n"
      " • 'What is ashwagandha coffee?'\n"
      " • 'Compare turmeric coffee vs saffron coffee'\n"
      " • 'I prefer low-caffeine drinks. Suggest something for me'\n"
      " • 'Typical brew ratio for beaten coffee?'\n"
```

```
      "Type 'exit' to quit.\n")

while True:
    q = input("You: ").strip()
    if q.lower() in {"exit", "quit"}:
        print("Goodbye!"); break
    _ = branched_rag_turn(q)
```

Adaptive RAG



1. Concept

1.1 What it is

Adaptive RAG is a retrieval-generation pipeline that **changes its behavior at runtime** based on the query, the state of retrieval, and operational goals (cost/latency/accuracy). Where **Branched RAG** routes by *intent* (explain, compare, recommend, lookup), **Adaptive RAG** continuously tunes the *knobs* within any branch: how much to retrieve, whether to re-rank, which filters to apply, which prompt to use, whether to include memory, and how hard to self-check.

1.2 Why adapt

A single static recipe (fixed top-k, always the same prompt, no re-rank) either wastes tokens on easy questions or under-serves hard ones. Adaptation reduces **cost/latency** on easy queries and boosts **recall/faithfulness** on difficult or ambiguous ones.

1.3 Signals you can read at runtime

- **Lexical cues:** query length, presence of "why/how/compare/suggest", digits/units (mg, ml, °C), named entities.
- **Retrieval health:** dispersion or "steepness" of similarity scores after an initial wide search (e.g., top-1 vs top-N gap), count of on-topic hits.
- **History & memory:** did the user state preferences earlier? did we store long-term facts that match this query?
- **SLOs:** target latency/cost budgets for this request type or user tier.

1.4 Knobs you can tune per query

- **k and candidate breadth:** small k for easy/precise; larger k for ambiguous.

- **Filters:** apply metadata filters when lookup-like tokens appear; relax them when recall is poor.
- **Re-ranking stage:** enable for hard queries; skip for easy ones.
- **Prompt style:** concise template for simple facts; structured comparison template for "vs"; recommendation template only when memory is relevant.
- **Self-check/fallback:** add a quick judge for low-confidence drafts; re-retrieve and revise if needed.

1.5 Control policies

Start with **heuristics** (transparent, teachable), progress to **score-based** thresholds (on retrieval steepness, hit quality), and graduate to **learned controllers** (bandits/trees) once you have logs. Always evaluate on **accuracy vs cost vs latency** and add safety checks for refusal/grounding.

2. Code

Assumptions: embeddings, Pinecone `index`, `embedder`, `llm`, `DOCS_NS`, optional memory namespace `MEM_NS`, and a `history` list already exist (from your Memory/Branched notebooks). Prompts live in `prompts/` as YAML files.

2.1 Templates (kept outside code)

`prompts/adaptive_lite.yaml`

```
name: adaptive_lite
template: |
  You are a coffee expert. Answer the QUESTION using only CONTEXT.
  Keep it under 80 words. If missing, reply exactly:
```

```
  "I don't know from the provided documents." Cite sources as
[DocID].

  Conversation so far:
  {history}

  QUESTION:
  {query}

  CONTEXT:
  {context}
```

prompts/adaptive_deep.yaml

```
name: adaptive_deep
template: |
  You are a coffee expert. Provide a thorough, well-structured answer
using only CONTEXT.
  If evidence is weak, say what is missing. Cite sources as [DocID].

  Conversation so far:
  {history}

  QUESTION:
  {query}

  CONTEXT:
  {context}
```

prompts/adaptive_recommend.yaml

```
name: adaptive_recommend
template: |
  You are a coffee expert. Make a personalized recommendation based
on RELEVANT_MEMORY,
  supported strictly by CONTEXT. If insufficient, ask one brief
clarifying question.
  Cite sources as [DocID].

  Conversation so far:
```

```
  {history}

  Relevant memory (facts about this user):
  {memory_block}

  QUESTION:
  {query}

  CONTEXT:
  {context}
```

2.2 Utilities (template loading, formatting)

```
import yaml, re, time

# Load YAML templates by name
TPLS = {
    "lite":      "prompts/adaptive_lite.yaml",
    "standard":  "prompts/adaptive_standard.yaml",
    "deep":      "prompts/adaptive_deep.yaml",
    "recommend": "prompts/adaptive_recommend.yaml",
}
def load_template(key: str) -> str:
    with open(TPLS[key], "r", encoding="utf-8") as f:
        return yaml.safe_load(f)["template"]

def format_history(history):
    if not history: return "None"
    return "\n".join(f"{'User' if h['role']=='user' else
'Assistant'}: {h['content']}" for h in history)

def format_context(hits):
    if not hits: return "None"
    lines = []
    for h in hits:
        doc_id = h.get("id","")
        meta = h.get("metadata",{})
        title = meta.get("title") or meta.get("file_name") or
"Untitled"
```

```
        lines.append(f"[{doc_id}] {title}")
    return "\n".join(lines)

def format_memory(mem_hits):
    if not mem_hits: return "None"
    return "\n".join(f"- {h['metadata'].get('fact','')}" for h in
mem_hits)
```

2.3 Signals and quick difficulty analysis

```
def embed_one(text: str):
    return embedder.encode([text],
normalize_embeddings=True)[0].tolist()

# Shallow lexical features
def lexical_signals(q: str) -> dict:
    ql = q.lower()
    return {
        "is_compare": bool(re.search(r"\bvs\b|compare|difference
between", ql)),
        "is_recommend": bool(re.search(r"\bsuggest|recommend|for me|i
prefer|what should i try", ql)),
        "has_units": bool(re.search(r"\b\d+
?(mg|ml|g|%|minutes|min|ratio|:|°c|celsius)\b", ql)),
        "is_why_how": bool(re.search(r"\bwhy|how\b", ql)),
        "len_tokens": len(re.findall(r"[A-Za-z0-9\-]+", ql)),
    }

# One wide probe to gauge retrieval "steepness" and ambiguity
def probe_retrieval(q: str, top_k: int = 12):
    vec = embed_one(q)
    res = index.query(vector=vec, top_k=top_k, include_values=False,
include_metadata=True, namespace=DOCS_NS)
    hits = res.get("matches", [])
    scores = [h["score"] for h in hits] if hits else []
    if not scores:
        return {"hits": [], "scores": [], "steep": 0.0, "mean": 0.0}
    # Steepness: top1 minus mean of remaining; larger ⇒ clearer best
```

```
doc
    steep = scores[0] - (sum(scores[1:]) / max(1, len(scores)-1)) if
len(scores) > 1 else scores[0]
    return {"hits": hits, "scores": scores, "steep": steep, "mean":
sum(scores)/len(scores)}
```

2.4 Policy: choose profile and knobs

```
def choose_profile(q: str):
    sig = lexical_signals(q)
    prb = probe_retrieval(q, top_k=12)

    # Base profiles
    profile = "standard"
    k_final = 6
    use_memory = False
    template_key = "standard"

    # Easy factual: short, units/lookup, or very steep retrieval
    if sig["has_units"] or (sig["len_tokens"] < 6 and prb["steep"] >
0.05):
        profile = "lite"; k_final = 4; template_key = "lite"

    # Compare or why/how usually need broader evidence
    if sig["is_compare"] or sig["is_why_how"]:
        profile = "deep"; k_final = 10; template_key = "deep"

    # Recommend uses memory (if available) + standard depth
    if sig["is_recommend"]:
        profile = "recommend"; k_final = 6; use_memory = True;
template_key = "recommend"

    # Ambiguous retrieval (low steepness) → deepen
    if prb["steep"] < 0.015 and profile != "recommend":
        profile = "deep"; k_final = max(k_final, 10); template_key =
"deep"
```

```
    knobs = {
        "profile": profile,
        "k": k_final,
        "use_memory": use_memory,
        "template_key": template_key,
        "probe_hits": prb["hits"],  # we can reuse some of them to
save an extra round-trip if desired
    }
    return knobs
```

2.5 Retrieval with light keyword filter (optional)

```
def retrieve_docs_adaptive(q: str, k: int):
    # Optional: if query has obvious tokens, try a small keyword
filter on headings
    kws = re.findall(r"[A-Za-z][A-Za-z\-]+", q)
    filt = {"heading_keywords": {"$in": [kw.lower() for kw in
kws[:6]]}} if kws else None
    vec = embed_one(q)
    res = index.query(vector=vec, top_k=k, include_values=False,
include_metadata=True, namespace=DOCS_NS, filter=filt)
    hits = res.get("matches", [])
    if not hits:
        # fallback to unfiltered
        res = index.query(vector=vec, top_k=k, include_values=False,
include_metadata=True, namespace=DOCS_NS)
        hits = res.get("matches", [])
    return hits
```

2.6 Memory helpers (reuse from Memory RAG)

```
def extract_user_facts(txt: str):
    pats = [r"\bI (?:like|love|prefer)\b[^.]+", r"\bI
(?:avoid|usually|often|am)\b[^.]+", r"\bMy [A-Za-z ]+\b[^.]+"]
    out = []
    for pat in pats:
        out += [m.group(0).strip() for m in re.finditer(pat, txt,
```

```
flags=re.I)]
    return sorted(set(out))

def add_memory_facts(facts: list):
    if not facts: return
    vecs = embedder.encode(facts, normalize_embeddings=True).tolist()
    now = int(time.time())
    payload = []
    for i, fact in enumerate(facts):
        vid = f"user_001:{now}:{i}"
        payload.append((vid, vecs[i], {"fact": fact,
"user_id":"user_001", "ts": now}))
    index.upsert(vectors=payload, namespace=MEM_NS)

def retrieve_memory(q: str, k: int = 3):
    vec = embed_one(q)
    res = index.query(vector=vec, top_k=k, include_values=False,
include_metadata=True, namespace=MEM_NS)
    return res.get("matches", [])
```

2.7 Build prompts (include memory only when used)

```
def build_prompt_from_tpl(tpl_key: str, query: str, history,
doc_hits, mem_hits=None):
    tpl = load_template(tpl_key)
    return tpl.format(
        history=format_history(history),
        query=query,
        context_block=format_context(doc_hits),
        memory_block=(format_memory(mem_hits) if mem_hits else
"None")
    )
```

2.8 One adaptive turn

```
history = []
```

```
def adaptive_rag_turn(user_text: str):
    # 1) collect any new user preferences into long-term memory
    add_memory_facts(extract_user_facts(user_text))

    # 2) choose knobs based on signals + probe retrieval
    knobs = choose_profile(user_text)

    # 3) final retrieval
    doc_hits = retrieve_docs_adaptive(user_text, k=knobs["k"])

    # 4) optional memory fetch
    mem_hits = retrieve_memory(user_text, k=3) if knobs["use_memory"]
else None

    # 5) build prompt per template
    prompt = build_prompt_from_tpl(knobs["template_key"], user_text,
history, doc_hits, mem_hits)

    # 6) LLM call
    resp = llm.generate_content(prompt)
    answer = getattr(resp, "text", "").strip()

    # 7) update history
    history.append({"role":"user", "content": user_text})
    history.append({"role":"assistant", "content": answer})

    # 8) trace for students
    print(f"[Adaptive profile: {knobs['profile']} | k={knobs['k']} |
mem={bool(knobs['use_memory'])}]")
    print("Top docs:", [h["id"] for h in doc_hits[:3]])
    print(answer, "\n")
    return answer
```

2.9 Quick demo loop

```
print("Adaptive RAG -- try:\n"
      " • 'What is masala coffee?'\n"
      " • 'Compare turmeric coffee vs saffron coffee'\n"
```

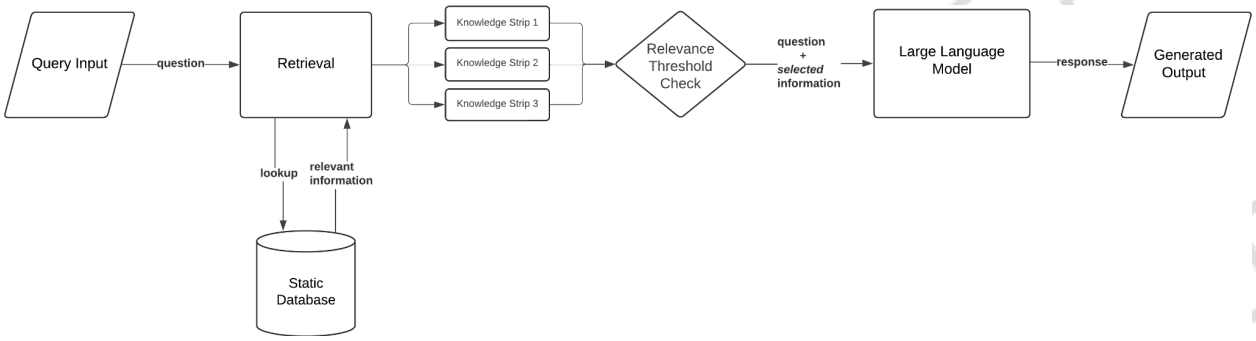
```
      " • 'I prefer low-caffeine drinks. Recommend something for
me'\n"
      " • 'Typical brew ratio for beaten coffee?'\n"
      "Type 'exit' to quit.\n")

while True:
    q = input("You: ").strip()
    if q.lower() in {"exit","quit"}:
        print("Goodbye!"); break
    _ = adaptive_rag_turn(q)
```

What to observe

- **Profile choices:** short lookups will show `lite`, comparisons/why/how will switch to `deep`, and preference queries to `recommend`.
- **k changes with difficulty:** deeper profiles fetch more context; easy profiles keep cost low.
  Memory inclusion is conditional: only recommendation-style queries bring in long-term facts.
- **Always grounded:** Pinecone is queried on every turn, regardless of profile.

Corrective RAG



1. Definition and Purpose

1.1 What it is

Corrective RAG is a **post-draft repair loop** that sits on top of a standard RAG pipeline. After you retrieve evidence and the LLM produces a draft answer, a validator checks the draft for **groundedness, coverage, and format**. If the draft fails, the system **corrects the retrieval or the answer** (or both) and tries again within a small, budget-bounded loop.

1.2 Why it exists

Even well-designed RAG pipelines miss the mark when the query is ambiguous, retrieval is off-topic, or the model overgeneralizes. Corrective RAG turns a brittle single-shot pipeline into a **self-healing** one that improves precision without permanently raising cost for every query.

1.3 Where it lives

The loop wraps the "retrieve → prompt → generate" core: **Draft → Validate → Repair Retrieval → Regenerate → Re-validate**, with strict stop conditions (max iterations, latency budget, token budget).

2. Failures Corrective RAG Targets

2.1 Retrieval failures

- **Empty or thin hits**: few passages or low scores.
- **Off-topic hits**: semantic neighbors that miss key entities (e.g., "ginger coffee" retrieved for "turmeric coffee").
- **Over-narrow filters**: aggressive metadata filters that exclude relevant pages.
- **Under-narrow filters**: noisy result sets that swamp the prompt.

2.2 Generation failures

- **Ungrounded claims**: statements not supported by any retrieved passage.
- **Citation drift**: sources present but irrelevant to sentences they are meant to support.
- **Coverage gaps**: multi-facet questions answered partially (e.g., compare X vs Y but only X discussed).
- **Format non-compliance**: missing citations, missing refusal when evidence is absent.

2.3 Detection signals

- **Retrieval side**: low top-k scores, small top-1 vs top-N gap (flat score curve), low diversity of doc IDs.
- **Answer side**: missing citations, citations that don't mention core entities, numeric claims without local evidence, template violations.

- **Semantic checks**: NLI/entailment between claims and passages, overlap of key query terms with cited text, per-facet coverage tests.

3. The Corrective Loop (What actually happens)

3.1 Validate the draft
A fast validator assigns: **E** (evidence adequacy), **F** (faithfulness/entailment), **C** (coverage of the question facets), and **R** (rule/format compliance). If any fall below thresholds, the draft is rejected for repair.

3.2 Repair retrieval

- **Broaden** when recall is low: raise k, relax filters, widen date/domain, add synonyms.
- **Narrow** when noise is high: apply metadata filters (e.g., heading keywords), re-rank, drop near-duplicates.
- **Switch retriever** when needed: dense ↔ lexical for identifiers, ratios, temperatures.
- **Targeted re-retrieve**: extract ungrounded claims and run focused sub-queries per claim ("turmeric latte caffeine content", "ashwagandha side effects").

3.3 Regenerate with pressure to ground
Regenerate with a template that **tightens citation rules** (e.g., "attach a [DocID] to each factual sentence" or "answer only with information supported by [DocID] passages below"). Keep prompts concise but stricter on attribution.

3.4 Stop conditions and escalation
Stop when the validator passes, or after **N** loops, or when the **latency/token budget** is exhausted. If still failing, **escalate**: apologize, ask a clarifying question, or hand off to a human. Corrective RAG **fails fast and safely** rather than spiraling.

4. Policies, Thresholds, and Budgets

4.1 Thresholds
Set explicit pass bars, for example: **E ≥ 0.7, F ≥ 0.8, C ≥ 0.7, R = pass**. Tune per domain. Retrieval-side preconditions (e.g., "≥3 unique docs above 0.35 cosine") prevent wasting LLM calls on barren evidence.

4.2 Iteration budgets

Bound the loop with **max_loops (e.g., 2)**, **latency p95 targets**, and **token caps** per section (history, context, memory). Budgeting makes corrective behavior predictable and production-friendly.

4.3 Confidence and calibration

Do **not** trust the model's self-reported confidence. Calibrate on validator scores and retrieval health signals. Maintain **pass@1** vs **pass@2** analytics to prove the loop adds value without runaway cost.

5. Correction Tactics (When and why to use each)

5.1 Query rewriting

When the draft shows missing facets or mis-parsed entities, rewrite the query with explicit entity names, synonyms, or local language variants ("kesar" ↔ "saffron"). This raises recall without changing the user's intent.

5.2 Retriever switching

Swap dense for lexical (or hybrid) when identifiers, numbers, and units matter (brew ratios, °C, mg). Switch back to dense for narrative or concept questions.

5.3 Re-ranking and deduplication

Two-stage pipelines (ANN → cross-encoder) lift precision. Deduplicate near-identical passages to free context for diverse evidence.

5.4 Per-claim retrieval

Break the draft into **atomic claims**, then retrieve evidence **per claim**. This targets gaps precisely and prevents context bloat.

5.5 Memory-aware corrections

If the draft ignores a user preference already in long-term memory ("I prefer low-caffeine"), run a **memory fetch** and regenerate with a recommendation template that must honor those facts.

5.6 Safety and policy

If the validator detects policy risk (e.g., medical claims), switch to a **safety template** that restricts scope, adds disclaimers, or refuses, rather than trying to "correct" into unsafe territory.

6. Relationship to Branched and Adaptive RAG

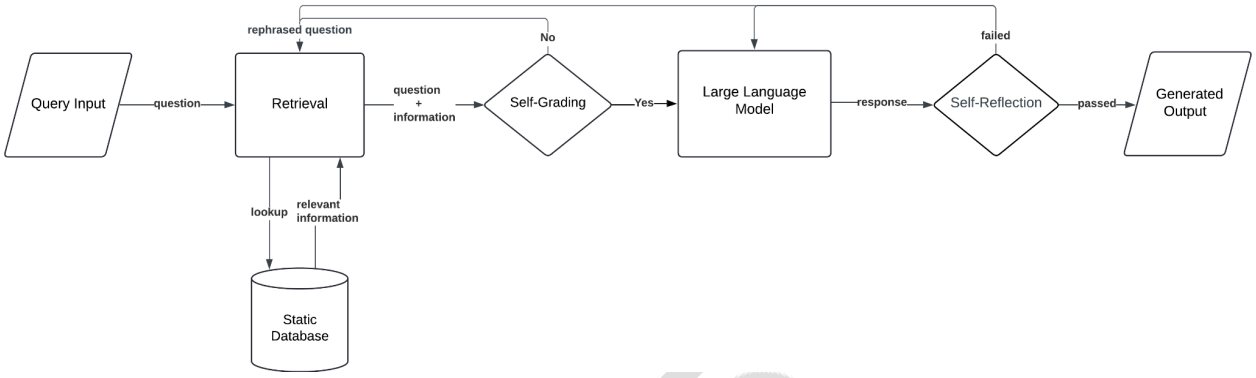
6.1 Complement, not replacement

Branched RAG chooses **which** recipe; Adaptive RAG tunes **how much** within that recipe; Corrective RAG **fixes what went wrong** after seeing evidence and a draft.

6.2 Escalation path

Typical production flow: **Branched (intent) → Adaptive (knobs) → Draft → Corrective (validate/repair)**. If corrective triggers often for a class of queries, push those learnings **upstream** into better routing and adaptive defaults.

Self RAG



1. Definition

Self-RAG is a form of **self-reflective retrieval-augmented generation**. Unlike traditional RAG, which retrieves once and generates once, Self-RAG equips the LLM with the ability to **critique its own draft** and decide whether to:

- Accept the answer as-is,
- Re-retrieve more evidence, or
- Regenerate with stricter grounding.

It essentially lets the model act as both **answerer** and **reviewer**, creating a feedback loop within a single query-response cycle.

2. Why Self-RAG Exists

1. **Hallucination reduction**: Even with RAG, models may invent facts. A self-check layer improves faithfulness.
2. **Dynamic evidence needs**: Some questions require more context; others are trivial. Self-RAG lets the model ask for "more docs" only when needed.
3. **Cost efficiency**: Instead of always fetching large top-k contexts, the system can start small, then expand only if the draft is weak.
4. **Better coverage**: The self-reflection stage can flag "missing aspects" (e.g., in a comparison query), triggering another retrieval before finalizing.

3. Core Components

3.1 **Initial retrieval and draft**

The model gets a modest set of passages (say top-5) and produces a first answer.

3.2 **Self-evaluation prompt**

The same model, or a smaller verifier, examines:

- Did I ground all claims in the retrieved context?
- Are any important parts of the question unanswered?
- Should I request more documents?

3.3 **Adaptive repair**

Based on the self-evaluation:

- If "yes, sufficient," output the draft.
- If "no, missing evidence," expand retrieval (e.g., fetch 10 more docs).
- If "contradictory evidence," regenerate with stricter citation rules.

3.4 **Loop control**

Limit to 1–2 reflection rounds to avoid runaway cost or infinite loops.

4. Benefits and Trade-offs

- **Benefits**: Improves **faithfulness**, reduces **hallucinations**, balances **recall vs cost**, handles complex/multi-part questions better.

- **Trade-offs**: Adds latency (extra round(s) of generation), increases token usage, and relies on the model's ability to critique itself (which is imperfect).

5. Industry Use Cases

- **Q&A systems at scale**: Self-RAG cuts hallucination without brute-force retrieval.
- **Customer support bots**: Reflection ensures sensitive answers (billing, compliance) are grounded before responding.
- **Research assistants**: Helps ensure multi-aspect queries (compare, analyze, summarize) are thorough and balanced.

6. Relation to Other RAG Variants

- **Memory-based RAG**: Focuses on continuity across turns; Self-RAG is about reflection within a single turn.
- **Branched RAG**: Routes by intent; Self-RAG adapts by self-critique.
- **Adaptive RAG**: Uses external heuristics to tune; Self-RAG uses internal model self-assessment.
- **Corrective RAG**: Relies on external validator to fix drafts; Self-RAG internalizes validation.

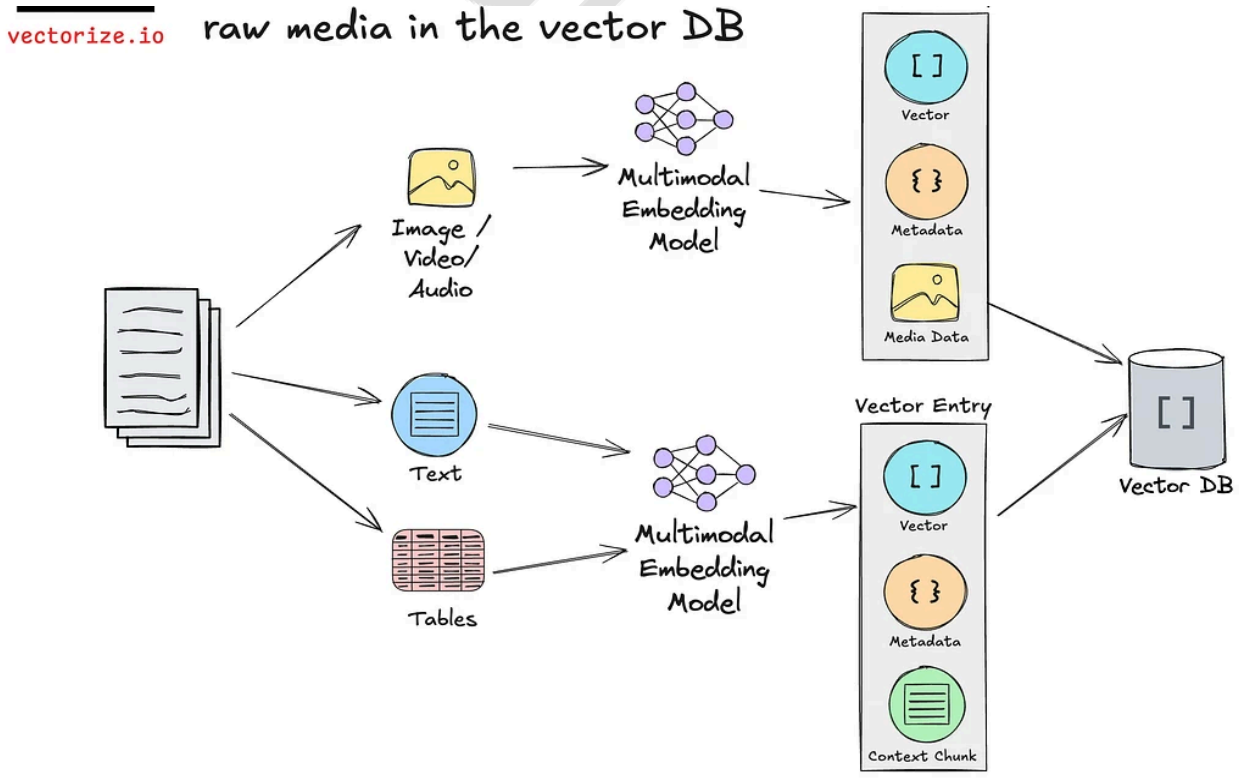
Comparative Table of RAG Variants

RAG Variant	Core Idea	When to Use	Industry Use Cases
Memory RAG	Stores and reuses user-specific facts (preferences, history) across turns by combining retrieval from both docs and memory vectors.	Needed when conversation spans multiple queries and personalization matters.	- E-commerce: remembering user's past purchases/preferences for recommendations.- Healthcare assistants: recalling patient's previous symptoms.- Meta-like workplace bots: remembering engineer's past tasks/queries.

Branched RAG	Routes queries into different retrieval+prompt pipelines (e.g., explain, compare, recommend, lookup).	When queries are heterogeneous and a single pipeline isn't enough.	- Search engines: different handling for definitions vs. comparisons.- Customer service: troubleshooting vs. FAQ vs. recommendations.- Meta internal knowledge search: explain design vs. compare experiments.
Adaptive RAG	Dynamically adjusts retrieval depth, filters, and prompts based on query difficulty and retrieval health signals.	When query load is high and you need to balance accuracy vs. latency vs. cost.	- Enterprise chatbots: simple FAQs answered cheaply, complex questions trigger deeper search.- Financial services: adaptively retrieving more docs for regulation-heavy queries.- Meta infra support bots: quick answers for easy metrics, deeper search for debugging.
Corrective RAG	Validates draft answers, then repairs retrieval or prompts if grounding is weak or incomplete (repair loop).	For high-stakes answers where hallucination or incomplete coverage is costly.	- Compliance/legal assistants: ensuring citations are present and correct.- Healthcare QA: no hallucinated advice, only grounded responses.- Meta production bots: patching incomplete answers about rollout timelines or SEVs.
Self-RAG	The model critiques its own draft, deciding to re-retrieve or regenerate if evidence is weak (self-reflection).	When self-checking improves quality without an external validator.	- Research assistants: ensuring coverage for multi-aspect queries.- Education platforms: model re-checks answers before giving to students.- Meta experimental bots: lightweight hallucination control without external validation infra.
Hybrid RAG	Combines multiple retrieval sources (vector search + keyword search + structured DB) to improve coverage.	When data is heterogeneous (unstructured text, structured tables, metadata).	- Search engines: hybrid BM25 + dense embeddings for relevance.- E-commerce: blending catalog DB filters with semantic search.- Financial research: mixing structured KPIs with reports.

Tool-Augmented RAG	Integrates external tools (APIs, calculators, code interpreters) alongside retrieved knowledge.	When queries require not just retrieval but external action or computation.	- Customer support: retrieving policies + checking account status via API.- Healthcare bots: retrieve medical literature + query drug interaction API.- Financial planning: retrieve docs + run tax calculator.
Multimodal RAG	Extends retrieval and grounding beyond text to include images, audio, video, or structured multimodal embeddings.	When knowledge spans beyond text and multimodal grounding is needed.	- Retail: product images + reviews for better recommendations.- Media: retrieving video transcripts + clips.- Healthcare: grounding answers in both radiology images and medical reports.

Multi-Modal RAG (Optional)



1. Simple Understanding

A normal RAG (Retrieval-Augmented Generation) works with **text only**: it pulls documents, finds relevant passages, and then the LLM uses those to answer.

But in the real world, information isn't only in text. Think of:

- **Pictures** (an MRI scan, a product photo, a screenshot)
- **Audio** (a doctor's voice note, a customer complaint recording)
- **Video** (training clips, surveillance footage)
- **Structured data** (tables, charts, KPIs)

Multimodal RAG means the system can **retrieve and ground answers using multiple kinds of data together**, not just text.

- If you ask *"What does this X-ray show?"*, it can pull **medical images + reports**.
- If you ask *"What's in this product video?"*, it can pull **transcripts + related specs**.
- If you ask *"Compare Apple's financial report with this chart"*, it can pull **both text from the report + the chart data**.

In simple words: **instead of only reading books, it can also look at pictures, listen to audio, and combine them all to answer better.**

2. How it Works

At a deeper level, **Multimodal RAG extends the retrieval pipeline** to support **cross-modal embeddings** and **alignment layers**:

2.1 Encoding Different Modalities

- Text → Sentence Transformers, LLM token embeddngs.
- Images → Vision encoders (CLIP, ViT, BLIP, SAM).
- Audio → Audio encoders (Whisper, Wav2Vec).
- Video → Frame-level encoders + temporal pooling.
  All of these convert raw signals into **vector embeddings** that live in the *same or aligned semantic space*.

2.2 Unified Vector Store (Multimodal Index)

- A vector database is extended to hold different modalities.
- Either with a **shared embedding space** (e.g., CLIP puts image and text together) or **multi-index hybrid retrieval** (text index + image index + metadata filters).

2.3 Retrieval Layer

- Queries can be multimodal too: e.g., "What is this image about?" (image → query embedding).
- Or text query + multimodal grounding: "Show me similar shoes to this picture" (cross-modal retrieval).

2.4 Fusion Strategies

- **Early Fusion**: All modalities are embedded into one space and retrieved together.
- **Late Fusion**: Retrieve separately from text, image, audio, then rank/merge results.
- **Hybrid Fusion**: Weighted combination depending on query type.

2.5 Augmented Generation

- The LLM receives retrieved evidence **as text plus references to non-textual modalities**.
- Example: an image is captioned/described using a vision model before feeding into the LLM.
- Advanced setups directly integrate vision-language models (e.g., Gemini, GPT-4o, Kosmos) that natively consume both text and images.

Applications

- **Healthcare**: Radiology → retrieve both scans + past reports for diagnostic grounding.
- **Retail**: Multimodal product search (image + description + reviews).

- **Education**: Explain a math problem by retrieving **text explanations + diagrams**.
- **Compliance**: Combine contracts (PDF text) + scanned signatures (image OCR).

Core Idea (Mature View):
Multimodal RAG isn't just about plugging in vision/audio models; it's about **aligning diverse representations into a retrieval+generation pipeline** so the LLM can reason across formats. The main challenges are **representation alignment, retrieval fusion, and multimodal grounding** during generation.

Conclusion:

1. From Foundations to Variants

- **Memory RAG** showed us how to maintain continuity and personalization across turns, ensuring user preferences and past interactions shape future answers.
- **Branched RAG** introduced intelligent routing, where queries are classified and handled by specialized sub-pipelines (e.g., explain, compare, recommend).
- **Adaptive RAG** demonstrated runtime flexibility, dynamically tuning retrieval depth, filters, and prompt styles based on query complexity and retrieval health.
- **Corrective RAG** layered in a repair loop, validating and fixing weak answers by repairing retrieval or tightening prompts, ideal for high-stakes use cases.
- **Self-RAG** closed the loop with model self-reflection, enabling the system to critique its own drafts and decide whether to re-retrieve or regenerate before delivering an answer.

2. Key Insights

- **Why so many types?** Each variant evolved because a single static pipeline cannot meet the diverse needs of industry-scale systems. Different users, tasks, and stakes require specialized adaptations.
- **Industry grounding:** We constantly linked these designs back to real-world scenarios—search engines, compliance, customer support, healthcare, and internal engineering support at companies like Meta.
- **Balance of trade-offs:** More robustness (e.g., Corrective RAG, Self-RAG) comes with extra latency and cost. Lighter pipelines (Adaptive RAG, Memory RAG) optimize efficiency. The art is in **choosing the right variant for the right problem**.

3. The Bigger Picture

Taken together, these RAG variants illustrate a trajectory:

- **Start simple** (basic retrieval + generation).
- **Add personalization and history** (Memory RAG).
- **Route by intent** (Branched RAG).
- **Tune dynamically** (Adaptive RAG).
- **Validate and repair** (Corrective RAG).
- **Let the model self-reflect** (Self-RAG).

This mirrors the path industry teams take when scaling retrieval systems—from basic prototypes to production-grade, reliable, and cost-aware pipelines.

4. Final Takeaway

RAG is not “one algorithm” but an **ecosystem of strategies**. Each variant represents a tool in the engineer’s toolbox, and the right combination depends on context: personalization, accuracy needs, latency budgets, and risk profile. By mastering these, students and practitioners can design **real-world AI agents** that move beyond static prompting into adaptive, self-improving systems that solve problems engineers at leading companies face daily.

codingninjas